

JARVIS

VOICE OS ASSISTANT

A modular, offline-first AI voice assistant for Windows 11

Manthan Dhanraj · B.Tech, Sharda University · v0.7.0

The Problem

1. **Cloud-bound** — The best assistants send your audio to remote servers.
2. **Offline-incapable** — They stop working the moment you lose connectivity.
3. **Not private** — Your voice leaves your device and your control.
4. **Closed** — You cannot inspect, extend, or truly own them.

The Solution

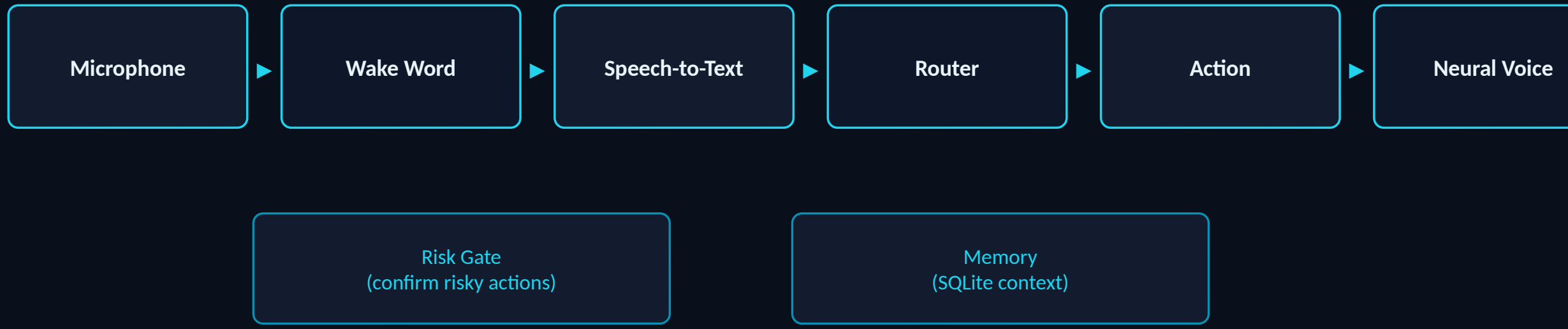
Talk to your PC. It listens, understands, acts, and replies — all on your machine.

- Wake word
- Local AI brain
- OS control



- Speech-to-text
- Neural voice
- Private & offline

How It Works — The Pipeline



Keyword commands are matched first; the local LLM handles everything else.

Architecture — 14 Modular Subsystems

core/

Lifecycle, session, router, risk

voice/

Mic, wake word, STT, TTS

brain/

Commands, intent, local LLM

automation/

Windows, web, dev, games, files

memory/

SQLite store & context

security/

Confirmation & risk gating

gui/

Tkinter dashboard + tray

config/ · utils/

Settings, logging, text

Voice I/O — Hearing You Reliably



Wake word: a gapless rolling-window listener spots "Hey Jarvis" continuously — no dead gap, no repeating.

Speech-to-text: faster-whisper, offline, with automatic GPU → CPU fallback so a missing driver never stops it.

Endpointing: energy-based capture starts on speech and ends on silence — you don't press anything.

3 design iterations turned wake-word activation from 'say it five times' into a single reliable phrase.

Speaking Back — A Human Voice

Three layered backends, tried in order — the voice never fails, it only changes quality.

1 Piper

Neural voice — natural & human-sounding

Preferred

2 SAPI5

Windows system voice via COM

Fallback

3 pyttsx3

Fresh engine per call

Last resort

The AI Brain — Natural Understanding

Local LLM via Ollama

qwen2.5 family, running entirely on your machine.

- Keyword commands are matched first — fast and predictable.
- Anything open-ended falls through to the local model.
- Speak naturally instead of memorising fixed phrases.
- If Ollama is offline, fixed commands keep working.



What Can It Do?

Apps	open notepad · close chrome
System	volume 30 · brightness 50 · mute
Web	open youtube · play a song · search
Coding	open project · run · create venv
Gaming	open steam · launch a game
Files	find resume · open downloads · organize
Memory	remember my roll number · recall it
Conversation	ask anything → local LLM

Safety by Design

SAFE	Apps, volume, web, search	Runs immediately
MEDIUM / HIGH	Destructive or system-level	Asks for confirmation
POWER	Shutdown / restart / sign-out	SAFE MODE — simulated unless armed

Risk is derived from the command category, not the model — so it can't be talked around.

Technology Stack

Python 3.14 Core language	**faster-whisper** Offline STT	**Piper** Neural TTS	**Ollama** Local LLM
sounddevice Audio capture	**SAPI5 / comtypes** System voice	**SQLite** Persistent memory	**Tkinter** Dashboard UI

Built & Hardened on Real Hardware

✕ GPU STT crash

→ Warm up at boot, auto-fallback to CPU

✕ Silent replies

→ Speak via SAPI directly; fresh engine per call

✕ Blocked media DLL

→ Stub the unused decoder; keep package importable

✕ Wake word missed

→ Callback-driven rolling-window capture

✕ Robotic voice

→ Integrate Piper neural voice

✕ Accidental shutdown

→ Power actions → SAFE MODE by default

Three Ways to Use It

Text

```
python main.py
```

Type commands — great for testing

Voice

```
--mode voice
```

Full hands-free with the wake word

GUI

```
--mode gui
```

Dashboard: status, chat, actions

Roadmap



1. **Voice authentication** Speaker verification (the deferred Day 6)
2. **GPU STT preset** Larger model on CUDA for higher accuracy
3. **Richer memory** Longer-horizon context & summarisation
4. **Packaged installer** One-click setup on a fresh machine
5. **Command plug-ins** User-defined automation domains



JARVIS

Private. Offline. Yours.

`github.com/manthandhanraj/Jarvis-OS`

Thank you · Manthan Dhanraj